

NOTES

Advance NumPy



Advanced Numpy concepts

✓ np.arange()

The np.arange() function in NumPy is used to create sequences of numbers with specified start, stop, and step values. It is similar to Python's range() but offers more flexibility, especially with floating-point numbers.

Syntax:

```
np.arange([start,] stop[, step], dtype=None)
```

Parameters

- start (optional): Starting value of the sequence (inclusive). Default is 0.
- stop (required): End value of the sequence (exclusive).
- step (optional): Spacing between values (default is 1).
- dtype (optional): Desired data type of the output array (inferred by default).

Examples

✓ Basic Usage

Generate an array from 0 (default start) to 9 (stop, exclusive):

```
import numpy as np
array1 = np.arange(10)
print(array1)
```

```
→ [0 1 2 3 4 5 6 7 8 9]
```

✓ Specifying Start and Stop

Create an array from 5 to 14:

```
array2 = np.arange(5, 15)
print(array2)
```

```
→ [ 5  6  7  8  9 10 11 12 13 14]
```

✓ Using Step

Generate an array with a step size of 2:

```
array3 = np.arange(0, 10, 2)
print(array3)
```

```
→ [0 2 4 6 8]
```

✓ Specifying Data Type

Generate an array with a specific data type (e.g., float):

```
array5 = np.arange(1, 6, dtype = float)
print(array5)
```

```
→ [1. 2. 3. 4. 5.]
```

✓ Combining Arrays

Combining arrays involves merging two or more arrays into one. This can be done along different axes (vertical or horizontal).

✓ Vertical Stacking (np.vstack):

This method stacks arrays along the vertical axis (row-wise), creating a new array where the original arrays are placed one below the other.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Vertical stacking
vstacked = np.vstack((a, b))
print("Vertically stacked:\n", vstacked)
```

```
➡ Vertically stacked:
[[1 2 3]
 [4 5 6]]
```

- np.vstack((a, b)) combines a and b vertically.
- The result is a 2D array where the first row is a and the second row is b.

✓ Horizontal Stacking (np.hstack)

This method stacks arrays along the horizontal axis (column-wise), creating a new array where the original arrays are placed side-by-side.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Horizontal stacking
hstacked = np.hstack((a, b))
print("Horizontally stacked:\n", hstacked)
```

```
➡ Horizontally stacked:
[1 2 3 4 5 6]
```

- np.hstack((a, b)) combines a and b horizontally.
- The result is a 1D array where elements of b follow elements of a.

✓ Column Stacking (np.column_stack)

This method combines 1D arrays into a 2D array by treating each array as a column.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Column stacking
cstacked = np.column_stack((a, b))
print("Column stacked:\n", cstacked)
```

```
➡ Column stacked:
[[1 4]
 [2 5]
 [3 6]]
```

- np.column_stack((a, b)) treats a and b as columns of a 2D array.
- The result is a 2D array where each element of a is paired with the corresponding element of b.

✓ Splitting Arrays

✓ np.split()

Splits an array into multiple sub-arrays along a specified axis.

```
arr = np.arange(1, 13).reshape(3, 4)

# Split into 3 subarrays along rows
split = np.split(arr, 3)
print("Split into subarrays:\n", split)

➦ Split into subarrays:
[array([[1, 2, 3, 4]]), array([[5, 6, 7, 8]]), array([[ 9, 10, 11, 12]])]
```

✓ Horizontal Splitting (np.hsplit)

Splits an array along the horizontal axis (columns).

```
arr = np.arange(1, 13).reshape(3, 4)

# Horizontal splitting
hsplit = np.hsplit(arr, 2)
print("Horizontally split arrays:\n", hsplit)

➦ Horizontally split arrays:
[array([[ 1,  2],
       [ 5,  6],
       [ 9, 10]])], array([[ 3,  4],
       [ 7,  8],
       [11, 12]])]
```

✓ Vertical Splitting (np.vsplit)

Splits an array along the vertical axis (rows).

```
arr = np.arange(1, 13).reshape(3, 4)

# Vertical splitting
vsplit = np.vsplit(arr, 3)
print("Vertically split arrays:\n", vsplit)

➦ Vertically split arrays:
[array([[1, 2, 3, 4]]), array([[5, 6, 7, 8]]), array([[ 9, 10, 11, 12]])]
```

✓ Numpy Statistical Functions

NumPy provides a wide range of statistical functions that make it easy to perform descriptive and inferential statistics on large datasets. These functions help in calculating measures of central tendency, dispersion, and other essential statistical values.

✓ Min and Max (np.min and np.max)

These functions return the minimum and maximum values in an array.

```
import numpy as np

data = np.array([3, 7, 1, 5, 9])
min_value = np.min(data)
max_value = np.max(data)
print("Min:", min_value)
print("Max:", max_value)

➦ Min: 1
   Max: 9
```

✓ Mean (np.mean)

The mean (average) is the sum of all elements divided by the number of elements. It gives an idea of the central value in a dataset.

```
import numpy as np

data = np.array([2, 4, 6, 8, 10])
mean_value = np.mean(data)
print("Mean:", mean_value)
```

➞ Mean: 6.0

✓ Median (np.median)

The median is the middle value of a dataset, which separates it into two equal halves. It's less affected by outliers than the mean.

```
import numpy as np

data = np.array([2, 4, 6, 8, 100]) # Note the outlier (100)
median_value = np.median(data)
print("Median:", median_value)
```

➞ Median: 6.0

✓ Standard Deviation (np.std)

Standard deviation measures the spread of data around the mean. A low standard deviation means the data points are close to the mean, while a high value indicates a wider spread.

```
import numpy as np

data = np.array([2, 4, 6, 8, 10])
std_dev = np.std(data)
print("Standard Deviation:", std_dev)
```

➞ Standard Deviation: 2.8284271247461903

✓ Variance (np.var)

Variance is the square of the standard deviation and measures the dispersion of data points. It's useful in determining how much the data varies.

```
import numpy as np

data = np.array([2, 4, 6, 8, 10])
variance_value = np.var(data)
print("Variance:", variance_value)
```

➞ Variance: 8.0

✓ Percentile (np.percentile)

Percentiles indicate the relative standing of a value within a dataset. For instance, the 25th percentile is the value below which 25% of the data falls.

```
import numpy as np

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
percentile_25 = np.percentile(data, 25)
percentile_50 = np.percentile(data, 50)
percentile_75 = np.percentile(data, 75)

print("25th Percentile:", percentile_25)
print("50th Percentile:", percentile_50)
print("75th Percentile:", percentile_75)
```

➞ 25th Percentile: 3.25
50th Percentile: 5.5
75th Percentile: 7.75

✓ Random Number Generation

The NumPy random module is used to generate random numbers in a variety of ways. This can be crucial for simulations, testing models, or creating randomized data for machine learning.

✓ rand() - Generating Random Numbers Between 0 and 1

The `np.random.rand()` function generates random numbers from a uniform distribution between 0 and 1.

```
import numpy as np
```

```
# Generate a single random number
random_number = np.random.rand()
print("Random Number:", random_number)
```

```
# Generate a 2x3 array of random numbers
random_array = np.random.rand(2, 3)
print("2x3 Random Array:\n", random_array)
```

```
➦ Random Number: 0.015643456255380772
2x3 Random Array:
[[0.30931445 0.3651977 0.23834195]
 [0.71662143 0.66010461 0.90182618]]
```

- `np.random.rand()` generates numbers between 0 and 1.
- If you provide dimensions, like (2, 3), it creates a 2x3 array of random numbers.

✓ randn() - Generating Random Numbers from a Normal Distribution

The normal (Gaussian) distribution is a continuous probability distribution where most values cluster around the mean. It's one of the most widely used distributions in statistics and machine learning. The `np.random.randn()` function generates random numbers from a normal distribution (mean = 0, standard deviation = 1).

```
import numpy as np
```

```
# Generate a 3x2 array of random numbers from a normal distribution
random_normal = np.random.randn(3, 2)
print("3x2 Random Normal Array:\n", random_normal)
```

```
➦ 3x2 Random Normal Array:
[[ 0.72482999  2.16607527]
 [-0.33026782 -1.47495983]
 [-0.76898419  1.45344066]]
```

`np.random.randn()` generates numbers with a mean of 0 and a standard deviation of 1, following a bell curve (normal distribution).

✓ randint() - Generating Random Integers

The `np.random.randint()` function generates random integers. You can specify the range and the size of the output.

- low is the lower bound of the random integers.
- high is the upper bound (exclusive).
- size specifies the shape of the output array.

```
import numpy as np
```

```
# Generate a single random integer between 0 and 10
random_int = np.random.randint(0, 10)
print("Random Integer:", random_int)
```

```
# Generate a 2x3 array of random integers between 1 and 100
random_int_array = np.random.randint(1, 100, (2, 3))
print("2x3 Random Integer Array:\n", random_int_array)
```

```
➦ Random Integer: 8
2x3 Random Integer Array:
[[44 53 67]
 [26 26 51]]
```

- `np.random.randint(0, 10)` generates integers between 0 and 10 (exclusive).
- `np.random.randint(1, 100, (2, 3))` generates a 2x3 array of integers between 1 and 100.

✓ Handling Missing Data in NumPy Arrays

Handling missing data is a common task in data processing, as real-world data often contains incomplete or missing values. NumPy, while not specifically designed for handling missing data (as this is more naturally handled in libraries like Pandas), provides several ways to manage missing values using placeholders like NaN (Not a Number) or masked arrays

✓ Identifying NaN Values :

The np.nan constant is used to denote missing values in arrays, and np.isnan() helps detect these values.

```
import numpy as np

# Create an array with missing values represented by NaN
data = np.array([1, 2, np.nan, 4, np.nan, 6])

# Check for NaN values
nan_mask = np.isnan(data)
print("NaN Mask:", nan_mask)
```

```
→ NaN Mask: [False False  True False  True False]
```

- This mask shows True where NaN values are present and False elsewhere.

✓ Removing Missing Values:

If you want to work with only the non-missing values, you can filter them out.

```
import numpy as np

# Remove NaN values from the array
data = np.array([1, 2, np.nan, 4, np.nan, 6])
data_no_nan = data[~np.isnan(data)] # ~ is negation sign
print("Data without NaN:", data_no_nan)
```

```
→ Data without NaN: [1. 2. 4. 6.]
```

✓ Filling Missing Values:

In some cases, it's useful to replace missing values with a specific number, such as the mean or median of the data, or a fixed value like 0.

```
import numpy as np

data = np.array([1, 2, np.nan, 4, np.nan, 6])

# Calculate the mean of non-NaN values
mean_value = np.nanmean(data)

# Replace NaN values with the mean
data_filled = np.where(np.isnan(data), mean_value, data)
print("Data with NaN filled:", data_filled)
```

```
→ Data with NaN filled: [1.  2.  3.25 4.  3.25 6. ]
```

- Here, np.nanmean(data) calculates the mean while ignoring NaN values, and np.where() replaces NaN values with the calculated mean.

Masked Operations:

Once data is masked, operations will only apply to valid (non-masked) elements.

```
# Perform operation on masked data (e.g., sum)
masked_sum = masked_data.sum()
print("Sum of Masked Data:", masked_sum)
```

```
→ Sum of Masked Data: 12.0
```

Operations like sum() only consider the valid data (ignoring NaN values).

THANK YOU



Stay updated with us!



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)